

8-Puzzle Problem

1. Aim

To implement the **8-Puzzle problem** using an Artificial Intelligence search technique to move the tiles from a given initial configuration to the desired goal configuration using the minimum number of moves.

2. Problem Statement

The 8-Puzzle problem consists of **eight numbered tiles and one blank space** arranged in a 3×3 grid. The objective is to rearrange the tiles from the initial state into the required goal state by moving the blank space **up, down, left, or right**. The solution should reach the goal state using an efficient sequence of moves.

3. Solution

The problem can be solved using the **A* search algorithm**, where each puzzle configuration is treated as a state. The algorithm uses the cost of moves already made and a heuristic estimate of the remaining moves to select the most promising state. The **Manhattan distance** of the tiles from their goal positions can be used as the heuristic function.

The evaluation function is:

$$f(n) = g(n) + h(n)$$

where **$g(n)$** is the cost of reaching the current state and **$h(n)$** is the estimated cost of reaching the goal state.

4. Algorithm

1. Start with the given initial 3×3 puzzle configuration.
2. Define the required goal configuration.
3. Calculate the heuristic value using Manhattan distance.
4. Add the initial state to the OPEN list.
5. Select the state having the lowest $f(n)$ value.
6. Find the position of the blank space.
7. Generate possible states by moving the blank space up, down, left, or right.
8. Calculate the cost and heuristic value for each new state.
9. Add unvisited states to the OPEN list.
10. Continue the process until the goal state is reached.
11. Trace the states to obtain the sequence of moves.
12. Display the solution path and number of moves.

5. Program

```
import heapq
```

```
def manhattan(state, goal):  
    distance = 0  
  
    for i in range(9):  
        if state[i] != 0:  
            current_row, current_col = divmod(i, 3)  
            goal_index = goal.index(state[i])  
            goal_row, goal_col = divmod(goal_index, 3)  
  
            distance += abs(current_row - goal_row)  
            distance += abs(current_col - goal_col)  
  
    return distance
```

```
def get_neighbors(state):  
    neighbors = []  
    zero = state.index(0)  
  
    row, col = divmod(zero, 3)
```

```
moves = [  
    (-1, 0),  
    (1, 0),  
    (0, -1),  
    (0, 1)  
]
```

```
for dr, dc in moves:
```

```
    new_row = row + dr
```

```
    new_col = col + dc
```

```
    if 0 <= new_row < 3 and 0 <= new_col < 3:
```

```
        new_zero = new_row * 3 + new_col
```

```
        new_state = list(state)
```

```
        new_state[zero], new_state[new_zero] = \  
            new_state[new_zero], new_state[zero]
```

```
        neighbors.append(tuple(new_state))
```

```
return neighbors
```

```
def a_star(start, goal):  
    priority_queue = []  
  
    h = manhattan(start, goal)  
  
    heapq.heappush(priority_queue, (h, 0, start))  
  
    cost = {start: 0}  
    parent = {start: None}  
  
    while priority_queue:  
        f, g, current = heapq.heappop(priority_queue)  
  
        if current == goal:  
            path = []  
  
            while current is not None:  
                path.append(current)  
                current = parent[current]
```

```
path.reverse()
```

```
return path
```

```
for neighbor in get_neighbors(current):
```

```
    new_cost = g + 1
```

```
    if neighbor not in cost or new_cost <
cost[neighbor]:
```

```
        cost[neighbor] = new_cost
```

```
        parent[neighbor] = current
```

```
    h = manhattan(neighbor, goal)
```

```
    f = new_cost + h
```

```
    heapq.heappush(
```

```
        priority_queue,
```

```
        (f, new_cost, neighbor)
```

```
    )
```

```
return None
```

```
# Initial state
```

```
start = (  
    1, 2, 3,  
    4, 0, 6,  
    7, 5, 8  
)
```

```
# Goal state
```

```
goal = (  
    1, 2, 3,  
    4, 5, 6,  
    7, 8, 0  
)
```

```
solution = a_star(start, goal)
```

```
print("Solution Path:\n")
```

```
for state in solution:  
    for i in range(0, 9, 3):  
        print(state[i:i+3])  
    print()
```

```
print("Number of Moves:", len(solution) - 1)
```

6. Result

The 8-Puzzle problem was successfully implemented using the **A* search algorithm** with the Manhattan distance heuristic. The program generates possible puzzle configurations and selects the most promising state until the goal configuration is reached.

Output:

Solution Path:

(1, 2, 3)

(4, 0, 6)

(7, 5, 8)

(1, 2, 3)

(4, 5, 6)

(7, 0, 8)

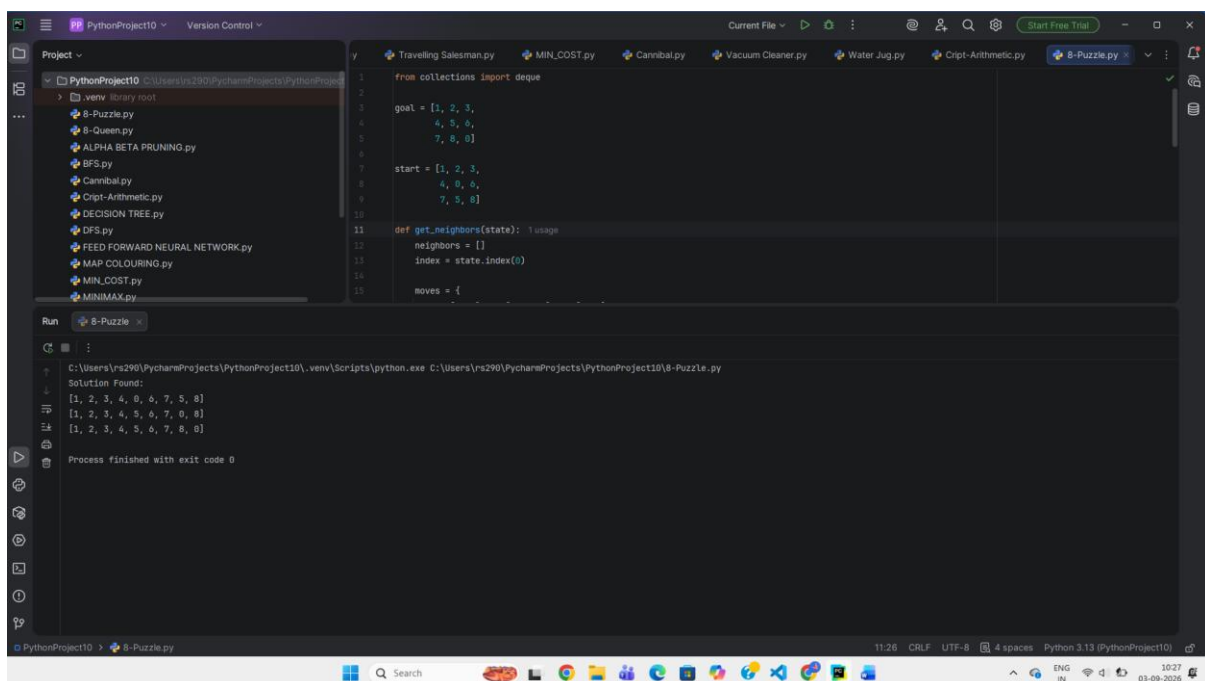
(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

Number of Moves: 2

Thus, the **8-Puzzle problem** was successfully solved in **2 moves**, demonstrating the use of the A* search algorithm for finding an efficient path from the initial state to the goal state.



The screenshot shows a Python IDE with a project named 'PythonProject10'. The file explorer on the left lists several files, including '8-Puzzle.py'. The main editor window displays the code for '8-Puzzle.py', which defines a goal state, a start state, and a function to get neighbors. The 'Run' console at the bottom shows the output of the program, indicating that the solution was found in 2 moves.

```
1 from collections import deque
2
3 goal = [1, 2, 3,
4         4, 5, 6,
5         7, 8, 0]
6
7 start = [1, 2, 3,
8         4, 0, 6,
9         7, 5, 8]
10
11 def get_neighbors(state):
12     neighbors = []
13     index = state.index(0)
14
15     moves = {
```

Run: 8-Puzzle

C:\Users\rs290\PycharmProjects\PythonProject10\.venv\Scripts\python.exe C:\Users\rs290\PycharmProjects\PythonProject10\8-Puzzle.py

Solution Found:

[1, 2, 3, 4, 0, 6, 7, 5, 8]

[1, 2, 3, 4, 5, 6, 7, 0, 8]

[1, 2, 3, 4, 5, 6, 7, 8, 0]

Process finished with exit code 0